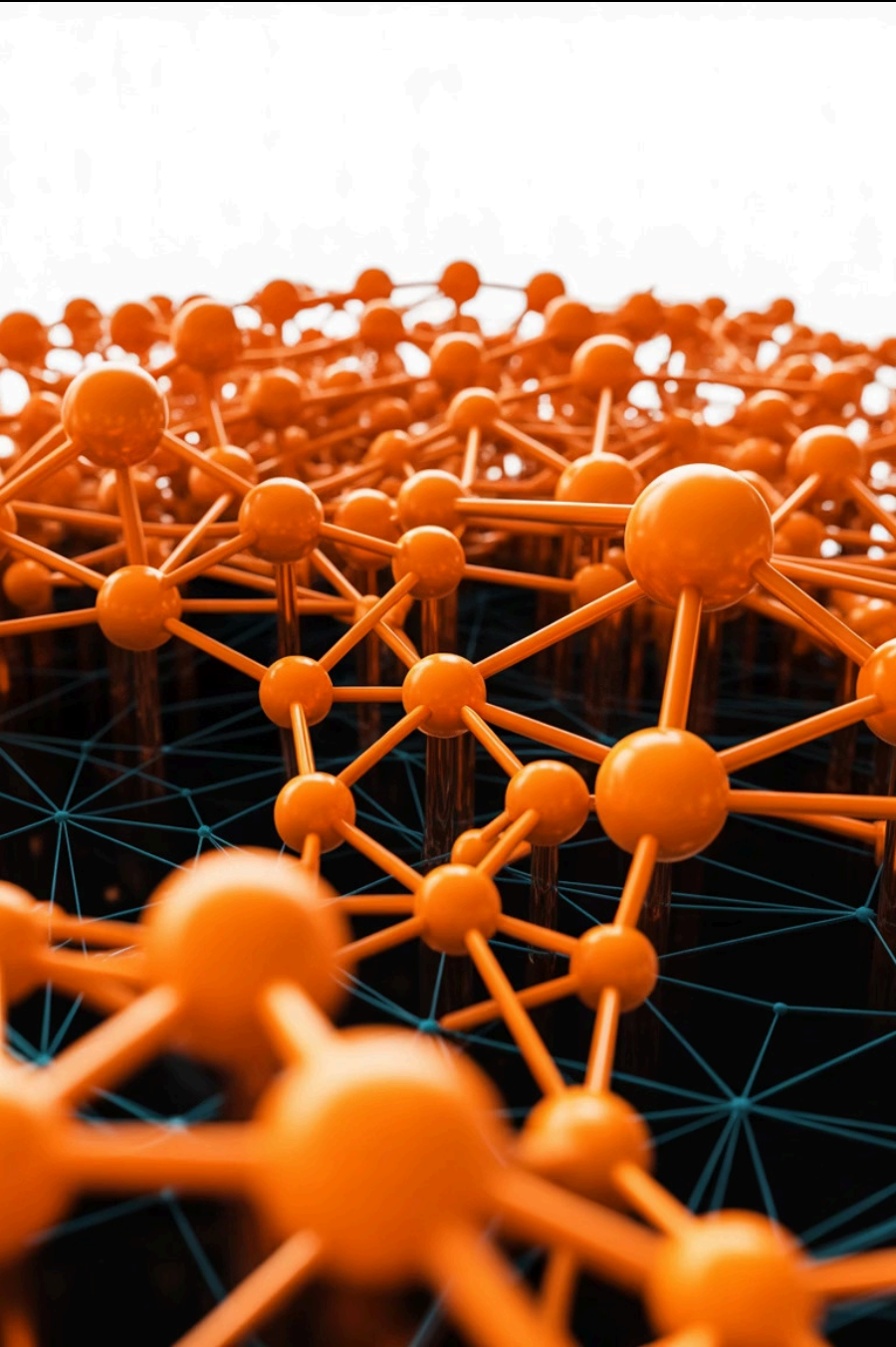




CO2030: Data Structures & Algorithms II

Lecture 1 (2h): Moving from Linear Thinking to Structured Thinking

Welcome to CO2030

CO2030 builds directly on the foundations you established in CO1030 – Data Structures & Algorithms I.

This module takes you beyond linear structures into the fascinating world of Trees, Graphs, and Hashing.

You've already mastered the basics—arrays, linked lists, stacks, and queues. Now we're going to explore how data can **branch**, **connect**, and **be retrieved** with remarkable **efficiency**.

These are the building blocks of every modern software system you use daily.



The Big Theme: A Journey Through Structure



Linear Structures

Arrays, linked lists—data in a single dimension



Hierarchical Structures

Trees—data with parent-child relationships



Networked Structures

Graphs—many-to-many connections



Fast Lookup Structures

Hashing—instant data retrieval

This progression mirrors how we naturally organise increasingly complex information. Each step unlocks new problem-solving capabilities that were impossible with simpler structures.

Why This Course Matters

The data structures you'll learn in CO2030 are the invisible infrastructure powering virtually every system you interact with. Understanding these structures transforms you from a code writer into a system architect.

Maps & Navigation

Graph algorithms calculate optimal routes in milliseconds

Compilers

Syntax trees parse and validate your code

AI Systems

Decision trees and graph neural networks

Databases

B-trees enable rapid data indexing and retrieval

File Systems

Directory trees organise billions of files

Quick Poll: Reflecting on DS&A I

Before we move forward, let's connect to what you've already learned. Which topic from Data Structures & Algorithms I resonated most with you?

Arrays

Direct access and contiguous memory

Linked Lists

Dynamic sizing and pointer manipulation

Stacks/Queues

LIFO and FIFO operations

Recursion

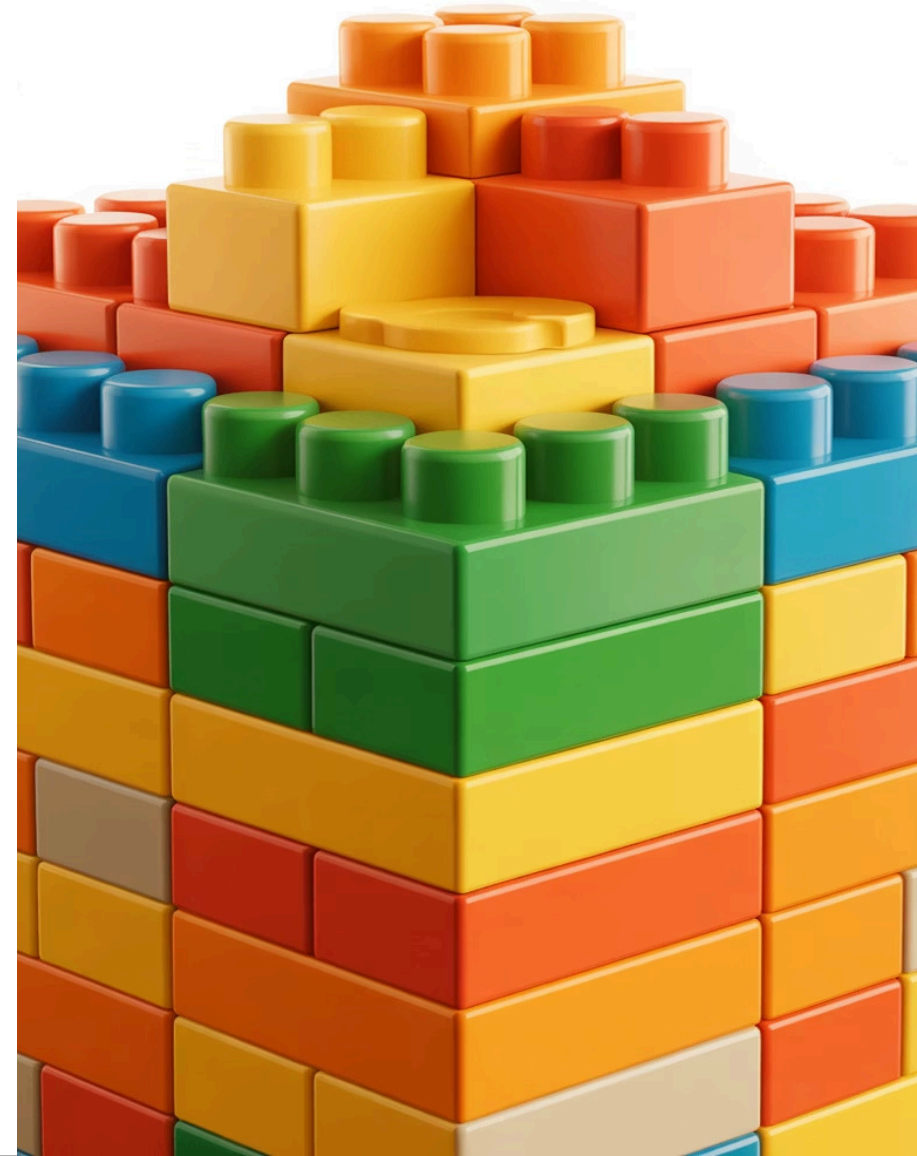
Self-referential problem solving

Sorting

Organising data efficiently

Your experience with these foundational structures provides the perfect springboard for understanding more complex data organisation.

Connecting to CO1030 -DS&A I



CO1030 - DS&A I Recap: What You Already Know

Your first data structures course established crucial foundations. You learned to work with fundamental structures and understand their performance characteristics. These are the lens through which you'll understand everything that follows.

Core Concepts Mastered:

- Arrays and their $O(1)$ access time
- Linked lists and dynamic memory
- Stacks and queues for ordered processing
- Recursion as a problem-solving technique
- Basic sorting algorithms and Big-O notation
- Searching in linear and sorted data



What These Structures Had in Common



One Dimension

Every structure stored data in a sequential line



Single Path

Only one way to traverse from start to end



Linear Relationships

Each element had at most one predecessor and successor

This linearity made these structures intuitive and easy to reason about. However, it also imposed **fundamental limitations** on the kinds of relationships we could model efficiently.

The Problem With Linear Thinking

Real-world data rarely organises itself into neat, sequential lines. Information branches, connects, and forms complex webs of relationships. When we try to force hierarchical or networked data into linear structures, we encounter serious problems.

Linear structures become inefficient or outright unusable when:

- Data has parent-child relationships (hierarchies)
- Multiple elements relate to the same item (many-to-many)
- We need to represent branching paths
- Relationships form cycles or networks

The fundamental issue: **linear structures collapse when relationships branch**. This limitation demands entirely new approaches to data organisation.





Example: The Family Tree Problem

Imagine trying to store your family tree using only an array. The problems emerge immediately:

Wasted Space

How do you handle different numbers of children? Pre-allocate maximum slots for each person? Most remain empty.

Sibling Representation

How do you indicate that multiple people share the same parents without duplicating parent data?

Dynamic Changes

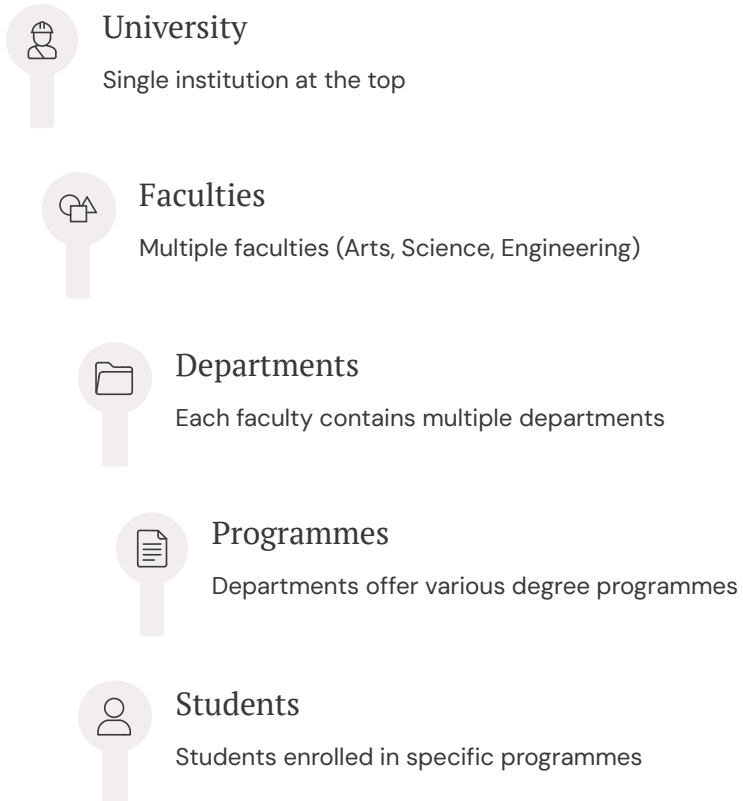
When someone has a child, how do you insert them without shifting massive portions of the array?

Navigation Complexity

Finding someone's grandparents requires complex index calculations that vary by family size.

A linked list doesn't help much either—the fundamental issue is that **linear structures can't elegantly represent branching relationships**.

Another Example: University Structure



This organisational hierarchy is natural to understand visually but nightmarish to represent with arrays or linked lists. You need a structure that inherently captures these multi-level, branching relationships. **A linked list can't represent this cleanly** without convoluted pointer schemes.



The Transition Point

When Linear Fails

The moment data relationships become **many-to-many** or **hierarchical**, linear structures show their limitations. We need structures that naturally express these relationships.

The next three months will fundamentally change how you think about organising and accessing information. Linear thinking got you here. Structured thinking takes you further.

Where We Go Next

This course begins precisely at this inflection point. We're moving from structures that force data into lines to structures that embrace complexity.

Course Introduction



Course Aim

To represent and solve complex computational problems efficiently using Trees, Graphs, and Hashing.

It's about developing the ability to look at a problem and immediately recognise which structure fits best. You'll learn to model real-world systems computationally and implement solutions that scale.

By the end of this module, you'll possess the skills that professional software engineers use to build high-performance systems.



Three Pillars of CO2030



Trees

Hierarchical structures for organising branching data. From binary trees to AVL trees, you'll master structures that naturally represent relationships with levels and inheritance.



Graphs

Networked structures for modelling complex connections. Learn to represent anything from social networks to road systems, and solve problems like shortest paths and connectivity.



Hashing

Fast lookup structures for instant retrieval. Discover how hash tables achieve near-constant-time operations and why they're fundamental to modern computing.

Each pillar unlocks new ways of thinking about problems. Together, they form the complete toolkit for advanced algorithmic problem-solving.

Intended Learning Outcomes

By successfully completing CO2030, you will be able to:

01

Describe and Analyse

Articulate the properties, strengths, and weaknesses of trees, graphs, and hash structures

02

Compare and Evaluate

Analyse different algorithms for the same problem and justify your choice based on time and space complexity

03

Model Problems

Translate real-world scenarios into appropriate data structure representations

04

Implement Solutions

Write efficient, correct code that leverages these structures to solve computational problems

These outcomes represent genuine capabilities that employers actively seek in graduates.



Weekly Structure



Lecture + Tutorial

2 hours weekly (unless announced otherwise)

New concepts introduced through explanation, examples, and live problem-solving. Interactive tutorials reinforce understanding.



Laboratory Sessions

2 hours weekly

Hands-on implementation practice. Ten practical slots (5 labs) throughout the term, each building on previous concepts.



Practical Tests

1 hour periodic

Five practical tests assessing your ability to implement solutions under time constraints.

This structure ensures consistent engagement with the material. Success requires showing up, coding, and thinking critically each week.

Assessment Breakdown

1

Practicals & Tests (50%)

Weekly practicals and five practical tests assess your implementation skills progressively throughout the term.

2

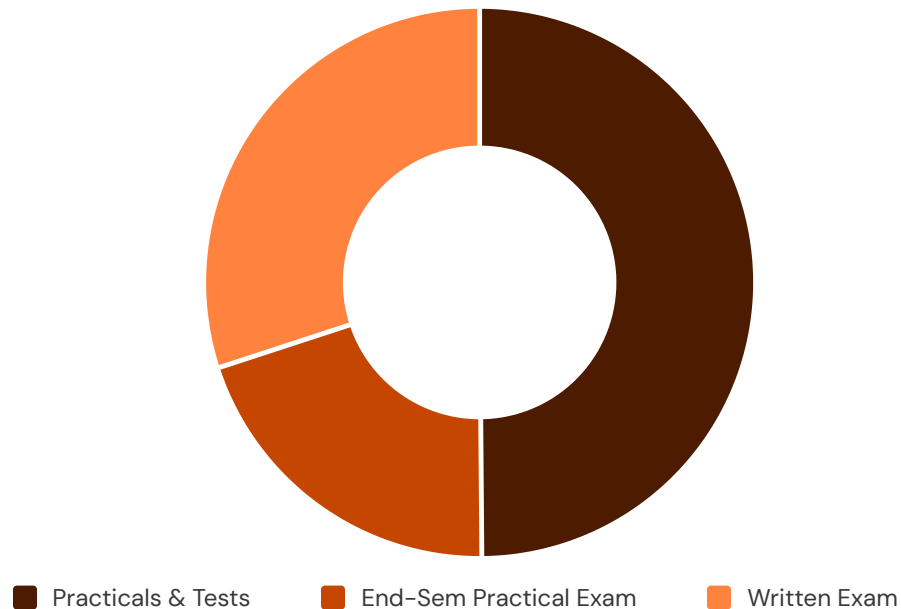
End-Semester Practical (20%)

A comprehensive coding examination testing your ability to implement solutions to novel problems.

3

Written Exam (30%)

Tests your theoretical understanding, algorithm analysis, and problem-modelling capabilities.



Practical Philosophy: How You Actually Learn DS&A



Drawing

Visualising structures on paper cements understanding. You'll sketch countless trees and graphs to internalise how they work.



Coding

Writing implementations reveals edge cases and deepens comprehension. Theory without practice remains abstract.



Debugging

Fixing broken code teaches you more than reading perfect solutions. Errors are learning opportunities.



Explaining

Articulating your logic to others (or yourself) exposes gaps in understanding and solidifies knowledge.

 **Important:** Online videos can't replace this active learning process. **Watching someone else code gives you the illusion of understanding.** Doing it yourself reveals what you actually know.

Tools You'll Use



VisualGo

Interactive algorithm visualisations that animate how data structures operate. Perfect for developing intuition before coding.



VPL/Moodle

Automated testing platform for submitting and validating your practical work. Immediate feedback on correctness.



GitHub Classroom (?)

Version control and code distribution system. Practicals distributed as repositories with starter templates.

Each practical comes with a simple template to get you started. Your job is to implement the core algorithms and data structure operations. These tools support your learning but don't replace the thinking required.



What You Can Expect From Lectures

- 1 — Short Explanations
Concise theoretical coverage of concepts and properties
- 2 — Concrete Examples
Real problems solved step-by-step
- 3 — Live Drawings
Visual representations built in real-time
- 4 — Practice (Interactive Q&A)
You work through problems with guidance

This isn't a passive experience. You'll be thinking, sketching, and problem-solving throughout each session. Bring paper, bring curiosity, and be ready to engage.

Mark Your Attendance

<https://shorturl.at/Sy0dZ>



Why DS&A Still Matters



DS&A Behind Every "Instant" Experience

Every time you experience something as "instant" on your device, data structures and algorithms are working invisibly in the background. These are the reason modern software feels fast and responsive.

Google Maps

Graph algorithms calculate optimal routes in milliseconds across millions of road segments

ChatGPT

Hash tables and tries index conversational context for rapid retrieval

Instagram Feeds

Graph structures model social connections to determine what content you see

Search Engines

Trees and hash tables enable instantaneous lookup across billions of web pages

Example: Google Maps

When you ask Google Maps for directions, you're triggering sophisticated graph algorithms that run in real-time. The road network becomes a graph where:

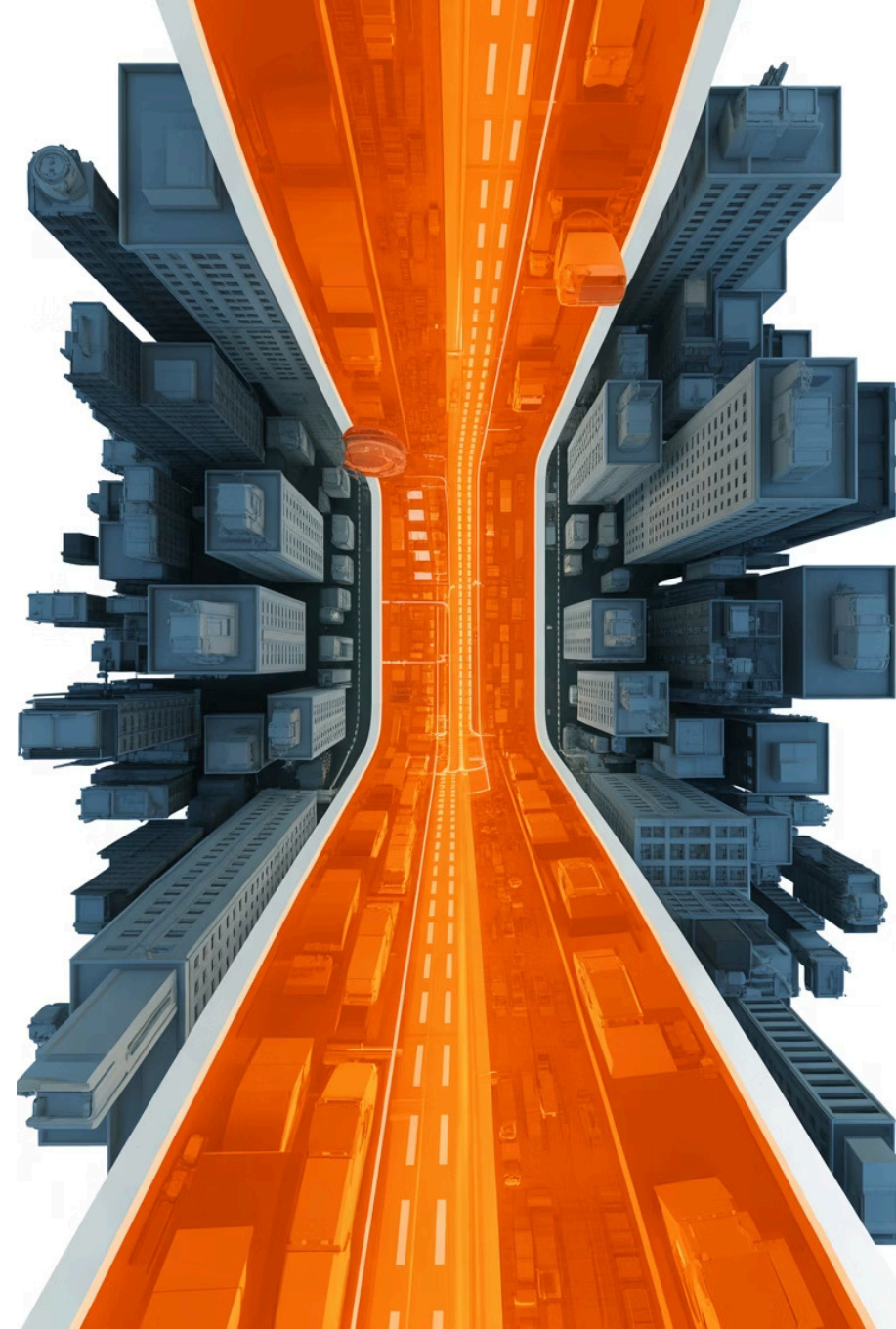
- Intersections are vertices (nodes)
- Road segments are edges
- Travel time or distance becomes edge weights

Algorithms like **Dijkstra's algorithm** and **A* (A-star)** search this massive graph to find the optimal path from your location to your destination. They consider one-way streets, traffic conditions, and road closures dynamically.

Why This Works

Graph structures naturally represent networks. The algorithms you'll learn can find shortest paths through millions of possible routes in fractions of a second.

Without proper data structures, this same calculation would take hours or be impossible entirely.



Example: ChatGPT



Behind ChatGPT's conversational abilities are multiple data structures working in concert:

Hash Tables

Rapid lookup of token embeddings and vocabulary mappings. Every word or word-piece gets instantly converted to its numerical representation.

Tries (Prefix Trees)

Efficient storage and retrieval of text patterns, enabling fast autocomplete and context matching.

Attention Graphs

Transformer models use graph-like structures to determine which parts of your conversation matter most for generating responses.

Your messages are not just processed linearly—they're indexed, hashed, and graphed to enable that instant, contextual reply.



Example: Your File Manager

Open any file manager on your computer. What you see is a direct visualisation of a tree data structure. Your entire file system *is literally a tree*.

Root Directory

The single starting point (/ on Unix, C:\ on Windows)

Directories as Nodes

Each folder is a node that can contain children

Files as Leaves

Files are leaf nodes—they don't contain other items

Paths as Tree Traversal

Every file path is literally a traversal from root to leaf

Operations like "find all files in this directory" are tree traversals. "How much space does this folder use?" requires recursively summing subtree sizes. Understanding trees means understanding your computer's fundamental organisation.

Example: Video Compression

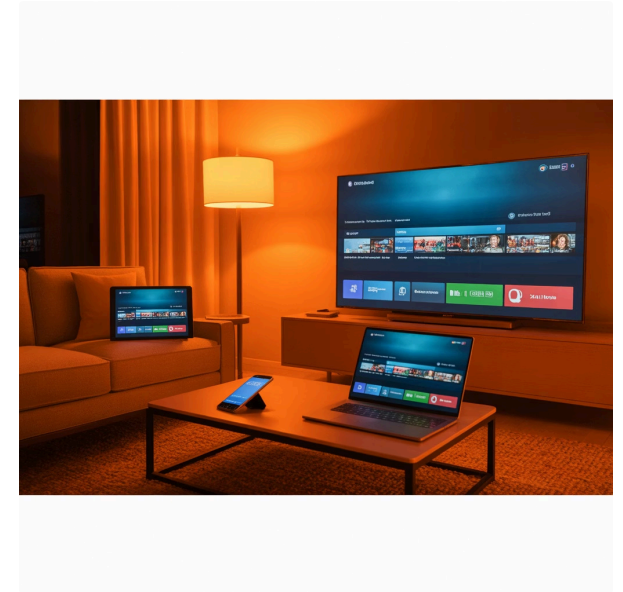
Huffman Coding Trees

Every time you watch a video or look at an image, Huffman coding is likely involved. This compression algorithm uses trees to represent data efficiently:

- Frequent characters get short codes
- Rare characters get longer codes
- The tree structure ensures no code is a prefix of another

This single algorithm, built entirely on tree structures, is used **billions of times each day** across image formats (JPEG), video codecs, and file compression (ZIP).

Without Huffman trees, your videos would be 2-3 times larger, streaming would require more bandwidth, and storage costs would skyrocket.





DS&A and Artificial Intelligence

AI didn't replace Data Structures & Algorithms. AI *runs on* Data Structures & Algorithms.

Neural Networks Use Graphs

Each neural network is a directed graph where neurons are nodes and connections are weighted edges. Training involves traversing this graph repeatedly.

Decision Trees in ML

Random forests and decision trees are literally tree structures used for classification and prediction.

Search Algorithms

AI planning and game playing use graph search algorithms you'll learn in this course.

Understanding data structures doesn't just complement AI—it's **prerequisite** to understanding how AI systems actually work under the bonnet.

A Simple Demo: VisualGo

Before we dive deeper into trees, let's see what we're building towards. Open VisualGo and navigate to the Tree Traversals visualisation.

 **Activity:** Visit visualgo.net → Binary Search Tree → Watch a single traversal animation

That one animation already demonstrates 90% of what we'll be building over the next few weeks:

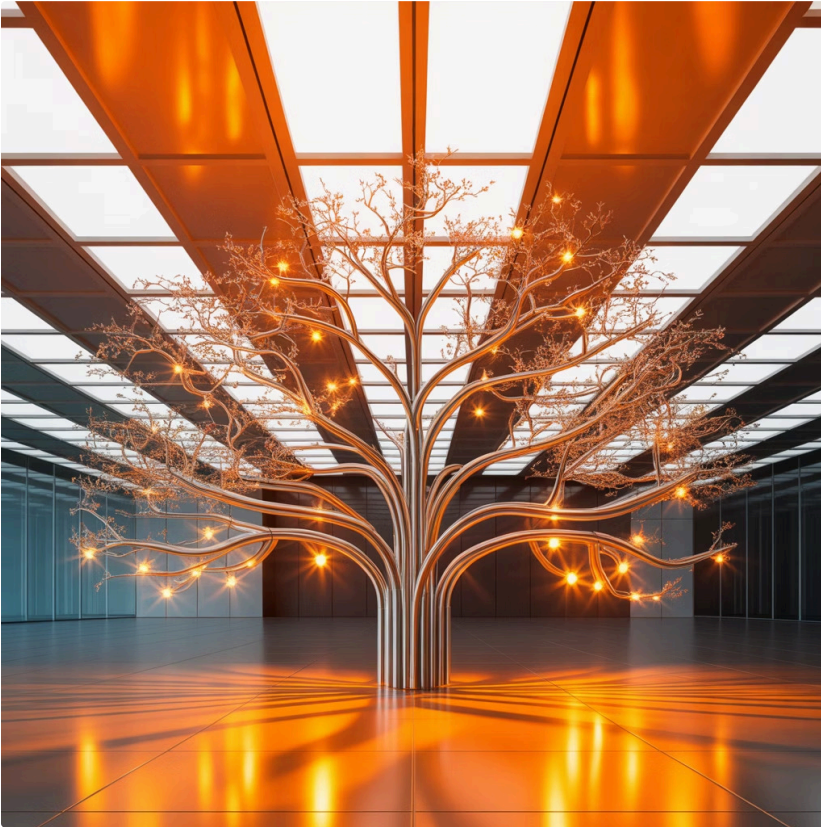
- How nodes connect in parent-child relationships
- How recursion naturally traverses tree structures
- Why order matters when visiting nodes
- How the same structure yields different outputs based on traversal strategy

Our goal is for you to understand this deeply enough to implement it from scratch and apply it to novel problems.

Starting the First Topic: Trees



Why Start With Trees?



Trees are the perfect bridge between linear and graph structures. They solve the fundamental problem that broke linear structures: **branching data**.

Trees represent hierarchy naturally:

- One parent, multiple children
- Clear top-to-bottom relationships
- Natural recursion in their structure

They're everywhere in computing, intuitive to visualise, and provide the perfect introduction to recursive thinking at a deeper level than you experienced in DS&A I.

Master trees, and graphs become a logical extension. Skip trees, and graphs feel overwhelming.



What Is a Tree?

A tree is a recursive structure: a node that points to zero or more subtrees.

This definition might seem circular, but it's precisely this recursive nature that makes trees powerful. Every node in a tree is itself the root of a smaller tree.

Key Properties

- Exactly one root (starting point)
- Each node has zero or more children
- No cycles—you can't loop back to a node
- Exactly one path between any two nodes

Why This Matters

The recursive definition means recursive algorithms feel natural. Processing a tree often means processing its root, then recursively processing each subtree.

Basic Tree Terminology

Root

The topmost node with no parent; the entry point to the tree

Parent

A node that has children below it

Child

A node directly below another node

Leaf

A node with no children; terminal nodes

Sibling

Nodes that share the same parent

Depth

Distance from the root to a node

Height

Longest path from a node to any leaf

Level

All nodes at the same depth

Real-World Examples of Trees



File Systems

Directories and files organised hierarchically. Your computer's storage is navigated as a tree.



Organisation Structures

Corporate hierarchies with managers and reports. Org charts are literally tree visualisations.



JSON/XML Structures

Nested data formats where elements contain other elements. Parsing these requires tree traversal.



Compiler Syntax Trees

Your code gets parsed into an abstract syntax tree before compilation.



Routing Tables

Network routers use trees to match IP addresses and determine packet destinations.



Search Indexes

Databases use B-trees and other tree structures to index data for rapid retrieval.

Recognising trees in the wild is the first step to using them effectively in your own designs.

Visual Example: A Simple Tree

Consider the tree structure (drawn on the board) with nodes containing integer values. Take a moment to identify:

- Which node is the root?
- Which nodes are leaves?
- What's the height of this tree?
- Which nodes are siblings?
- What's the depth of the deepest node?
- How many children does the root have?
- Can you identify all nodes at level 2?

Being able to quickly analyse a tree's structure is fundamental to understanding how algorithms will process it. Practice visualising and mentally traversing trees until it becomes second nature.



The Recursive Nature of Trees

"A tree is a node plus zero or more subtrees."

This recursive definition isn't just elegant—it's **practical**. It means every operation on a tree can be expressed recursively:

- **Count nodes:** 1 (for root) + count of all subtree nodes
- **Find height:** 1 + maximum height of any subtree
- **Search:** Check root, then search each subtree

The base case is always simple: an empty tree (null pointer). The recursive case processes the current node and recurses on children.

This is why recursion feels natural with trees—**the data structure itself is recursive**, so recursive algorithms align perfectly with its structure.

Tree Representation in Code

In C, we represent a tree node simply as a structure containing data and pointers to children. For binary trees (trees where each node has at most two children), the structure looks like this:

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

Breaking It Down

- `data` holds the node's value
- `left` points to the left child (or `nullptr`)
- `right` points to the right child (or `nullptr`)

Why This Works

Each node becomes a "portal" to smaller trees. Following a pointer takes you to the root of a subtree. This simple structure enables powerful recursive algorithms.

Why This Structure Works So Well

The beauty of this simple node structure is that **each node becomes a portal to a smaller tree**. This aligns perfectly with our recursive definition.

1

Every Node Is a Tree

Any node pointer you hold is actually pointing to the root of a valid tree (possibly with just one node, possibly with thousands)

2

Recursion Feels Natural

Because the structure is recursive, recursive code mirrors the data structure directly

3

Null Pointers End Recursion

When you reach a nullptr, you've found a base case—no more subtree to process

This is why recursion feels natural with trees in a way it doesn't with arrays or linked lists. The data structure itself demands recursive thinking.

Traversing a Tree: Four Main Approaches

Once you have a tree structure, the fundamental operation is **traversal**—visiting every node in some order. For binary trees, there are four standard traversal orders:

Preorder

Visit root first, then left subtree, then right subtree

Root → Left → Right

Inorder

Visit left subtree first, then root, then right subtree

Left → Root → Right

Postorder

Visit left subtree first, then right subtree, then root

Left → Right → Root

Level-order

Visit nodes level by level, left to right

Also called breadth-first traversal

The first three are depth-first traversals (dive deep before moving across). Level-order is breadth-first (process one level completely before moving down).

Wrap-Up and Next Steps

What We Covered Today

- Why linear structures have limits
- Course structure and expectations
- Real-world applications of DS&A
- Introduction to trees and their properties
- Basic tree terminology and representation

Next Week

- **Binary Trees** in depth
- **Binary Search Trees** and their properties
- **Practical 1:** Implementing tree traversals
- Hands-on coding with tree structures

This course is not about coding harder. It's about thinking deeper.

Come prepared to engage, sketch structures, write code, and challenge your assumptions. See you next week.